

CliniqAI Phase 1

Complete Guide: FastAPI + Server Logic

Learn FastAPI from zero
Understand `server.py` line by line
See real examples in action

Google Cloud Rapid Agent Hackathon · May 2026

PART 0: FastAPI Basics

Everything you need to know to understand server.py

What is FastAPI?

■ LAYER 1 — The Story

Imagine a receptionist at a clinic. Doctors walk up and say 'I have a photo' or 'show me Ramesh's history'. The receptionist understands them, does the work, and hands back an answer. FastAPI is that receptionist — it listens for requests from the outside world and routes them to the right function.

■ LAYER 2 — Technical

FastAPI is a Python web framework that:

- Listens for HTTP requests (GET, POST, etc.)
- Routes requests to the right function
- Validates incoming data automatically
- Returns JSON responses
- Generates interactive API documentation

Why FastAPI over Flask or Django?

- **Async support:** Handle multiple requests at the same time
- **Auto validation:** Pydantic checks data types automatically
- **Auto docs:** /docs page generated automatically
- **Fast:** One of the fastest Python frameworks

■ LAYER 3 — Example

Basic structure:

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/hello")
def hello():
    return {"message": "Hello, World!"}

# Run with: uvicorn main:app --reload
```

HTTP Methods: GET vs POST

■ LAYER 1 — The Story

GET is like asking a librarian 'show me books about Python'. POST is like handing the librarian a form and saying 'store this information for me'. GET retrieves data. POST sends data.

■ LAYER 2 — Technical

GET: Retrieve data (no body)

```
@app.get("/patients") def get_patients(): return {"patients": [...]} # Call with: GET http://localhost:8000/patients
```

POST: Send data (with body)

```
@app.post("/process") def process_image(file: UploadFile): return {"status": "processed"} # Call with: POST http://localhost:8000/process # With file attachment in body
```

■ LAYER 3 — Example

Real examples from CliniqAI:

```
GET /health → Returns: {"status": "running", "mongodb": "connected"} POST /process → Accepts: Image file → Returns: {"patient": {...}, "alerts": [...]} POST /query → Accepts: {"query": "how many patients"} → Returns: {"answer": "Total patients: 5"}
```

Decorators: @app.get() and @app.post()

■ LAYER 1 — The Story

A decorator is like a label on a mailbox. @app.get('/health') says 'when someone sends a GET request to /health, run this function'. The @ symbol is Python syntax for decorators.

■ LAYER 2 — Technical

Decorator syntax:

```
@app.get("/endpoint") def function_name(): return {"data": "here"} # The @ symbol means: 'apply this decorator' # app.get() is the decorator # It tells FastAPI: 'when GET /endpoint, run function_name' # The function name doesn't matter – the path does
```

Multiple decorators on same function:

```
@app.get("/patients") @app.get("/get-patients") # Both URLs work! def get_patients(): return {"patients": [...]}
```

■ LAYER 3 — Example

From server.py:

```
@app.get("/health") def health(): return {"status": "running"} @app.post("/process") async def process_document(file: UploadFile): return {"result": "..."} @app.post("/query") async def run_query(request: QueryRequest): return {"answer": "..."}
```

Request and Response Flow

■ LAYER 1 — The Story

A request is what the client sends. A response is what the server sends back. FastAPI automatically converts Python dictionaries to JSON.

■■ LAYER 2 — Technical

Request → Function → Response

```
# Client sends: POST /query {"query": "how many patients"} # FastAPI receives and converts to Python: request = QueryRequest(query="how many patients") # Function processes: def run_query(request: QueryRequest): answer = f"Total: {count_patients()}" return {"answer": answer} # FastAPI converts back to JSON and sends: {"answer": "Total: 5"}
```

Data validation with Pydantic:

```
from pydantic import BaseModel class QueryRequest(BaseModel): query: str # If client sends: {"query": "hello"} # FastAPI accepts ✓ # If client sends: {"query": 123} # FastAPI rejects X (query must be string) # If client sends: {"wrong_field": "hello"} # FastAPI rejects X (missing required field)
```

■ LAYER 3 — Example

Example from CliniqAI:

```
Client sends: POST /query {"query": "how many patients"} FastAPI validates: ✓ Is "query" a string? YES ✓ Is "query" present? YES Calls function: run_query(request=QueryRequest(query="how many patients")) Function returns: {"answer": "Total patients: 5"} Client receives: {"answer": "Total patients: 5"}
```

Async: Handle Multiple Requests

■ LAYER 1 — The Story

Imagine a restaurant. A synchronous waiter serves one table, waits for them to finish, then serves the next table. An async waiter takes an order from table 1, moves to table 2 while table 1's food is cooking, takes their order, and so on. Async means 'don't wait for slow operations'.

■■ LAYER 2 — Technical

Sync (blocking):

```
@app.post("/process") def process_image(file: UploadFile): image_bytes = file.read() # WAIT  
for file to load result = gemini_api_call(image_bytes) # WAIT for API return result # If  
API takes 5 seconds, user waits 5 seconds # Other requests are blocked!
```

Async (non-blocking):

```
@app.post("/process") async def process_image(file: UploadFile): image_bytes = await  
file.read() # Don't wait, continue result = await gemini_api_call(image_bytes) # Don't wait  
return result # If API takes 5 seconds, FastAPI handles other requests! # Multiple users  
can upload simultaneously
```

■ LAYER 3 — Example

In CliniqAI:

```
@app.post("/process") async def process_document(file: UploadFile = File(...)): # async =  
this function can run concurrently # Multiple doctors can upload images at same time  
image_bytes = await file.read() # await = wait for file to load, but don't block others  
extracted = extract_from_prescription(image_bytes) # This is fast (just calls our function)  
return {...}
```

PART 1: server.py Explained

Using FastAPI concepts to understand the code

Section 1: Imports

Import	What it does	Why we use it
os	Access environment variables	Read API keys from .env
re	Regular expressions	Escape special chars in queries
uuid	Generate unique IDs	Create unique patient IDs
datetime, date	Date/time handling	Record visit dates
FastAPI	Web framework	Create the server
UploadFile, File	File upload handling	Accept image uploads
CORSMiddleware	Cross-Origin requests	Allow frontend to call backend
BaseModel	Data validation	Validate incoming JSON
MongoClient	MongoDB connection	Connect to database
load_dotenv	Load .env file	Read API keys

Section 2: Creating the FastAPI App

■ LAYER 1 — The Story

We create one FastAPI app object. All endpoints are attached to this object. It's like creating a restaurant, then adding different counters (endpoints) to it.

■ LAYER 2 — Technical

```
app = FastAPI(title="CliniqAI", version="1.0") app.add_middleware( CORSMiddleware,
allow_origins=["*"], allow_methods=["*"], allow_headers=["*"], )
```

What this does:

- **FastAPI()** — Create the app
- **title, version** — Shown in /docs page
- **CORSMiddleware** — Allow requests from any frontend
- **allow_origins=["*"]** — Accept requests from anywhere

■ LAYER 3 — Example

Why CORS?

Without CORS: Frontend at localhost:3000 tries to call localhost:8000 Browser blocks it! ✗
With CORS: Frontend at localhost:3000 can call localhost:8000 Browser allows it! ✓

Section 3: Pydantic Models (Data Validation)

■ LAYER 1 — The Story

A Pydantic model is a blueprint for incoming data. It says 'I expect a JSON object with these fields and these types'. FastAPI automatically validates and rejects bad data.

■■ LAYER 2 — Technical

```
from pydantic import BaseModel class QueryRequest(BaseModel): query: str # This means: # -  
Expect a JSON object # - Must have a "query" field # - "query" must be a string # - Nothing  
else is allowed
```

Validation examples:

```
# ✓ ACCEPTED {"query": "how many patients"} # ✗ REJECTED (query is number, not string)  
{"query": 123} # ✗ REJECTED (missing required field) {"name": "Ramesh"} # ✗ REJECTED (extra  
field) {"query": "hello", "extra": "field"}
```

■ LAYER 3 — Example

In server.py:

```
@app.post("/query") async def run_query(request: QueryRequest): # FastAPI automatically: #  
1. Receives JSON # 2. Validates it matches QueryRequest # 3. Converts to Python object # 4.  
Passes to function query = request.query # Access the field return {"answer": "..."}
```

Section 4: /health Endpoint

■ LAYER 1 — The Story

The health endpoint is like a heartbeat check. Clients call it to see if the server is alive and if MongoDB is connected.

■■ LAYER 2 — Technical

```
@app.get("/health") def health(): db_connected = get_db() if db_connected: mongo_status = "connected" else: mongo_status = "not configured (using in-memory store)" return { "status": "running", "mongodb": mongo_status, "google_api_key_set": bool(GOOGLE_API_KEY and "your_google" not in GOOGLE_API_KEY), }
```

Line by line:

- **@app.get('/health')** — Create GET endpoint
- **def health()** — Function name (doesn't matter)
- **get_db()** — Try to connect to MongoDB
- **return {...}** — Return JSON response

■ LAYER 3 — Example

Example request/response:

```
REQUEST: GET http://localhost:8000/health RESPONSE: { "status": "running", "mongodb": "not configured (using in-memory store)", "google_api_key_set": false }
```

Section 5: /process Endpoint (The Main One)

■ LAYER 1 — The Story

This is the heart of CliniqAI. It receives an image, extracts patient data, checks for drug conflicts, and stores everything.

■■ LAYER 2 — Technical

```
@app.post("/process") async def process_document(file: UploadFile = File(...)): # Step 1: Read image image_bytes = await file.read() # Step 2: Extract data with Gemini extracted = extract_from_prescription(image_bytes) # Step 3: Check if patient exists patient_name = extracted.get("patient_name", "Unknown") existing_patient = find_patient_in_db(patient_name) # Step 4: Store or update if existing_patient: update_patient(existing_patient, extracted) else: insert_new_patient(extracted) # Step 5: Check drug conflicts alerts = check_drug_conflicts(...) # Step 6: Return result return {"patient": {...}, "alerts": alerts}
```

Key parameters:

- **file: UploadFile** — The uploaded image
- **= File(...)** — This is required (... means 'required')
- **async** — Handle multiple uploads simultaneously
- **await file.read()** — Wait for file to load

■ LAYER 3 — Example

Example request/response:

```
REQUEST: POST http://localhost:8000/process [Image file attached] RESPONSE: { "record_id":  
"507f1f77bcf86cd799439011", "is_returning": false, "patient": { "name": "Ramesh Gupta",  
"age": 45, "medicines": [...] }, "alerts": [ { "severity": "HIGH", "type": "ALLERGY",  
"message": "..."} ] }
```

Section 6: /query Endpoint

■ LAYER 1 — The Story

This endpoint accepts natural language questions and returns answers. For Phase 1, it does simple keyword matching.

■■ LAYER 2 — Technical

```
@app.post("/query") async def run_query(request: QueryRequest): query =  
request.query.lower() if "how many" in query or "count" in query: count = count_patients()  
return {"answer": f"Total patients: {count}"} elif "diabetic" in query or "diabetes" in  
query: patients = find_patients_with("diabetes") return {"answer": f"Found {len(patients)}  
diabetic patients"} else: # Search by medicine, condition, or name results =  
search_patients(query) return {"answer": f"Found {len(results)} result(s)"}
```

How it works:

- **request: QueryRequest** — Receives validated JSON
- **query.lower()** — Convert to lowercase for matching
- **if 'how many' in query** — Simple keyword matching
- **return {'answer': ...}** — Return natural language response

■ LAYER 3 — Example

Example requests/responses:

```
REQUEST 1: POST /query {"query": "how many patients"} RESPONSE: {"answer": "Total patients:  
5"} --- REQUEST 2: POST /query {"query": "metformin"} RESPONSE: {"answer": "Found 2  
result(s): Ramesh Gupta, Sunita Devi"}
```

CliniqAI Phase 1 Complete Guide · FastAPI + server.py

Ready for Phase 2: Build the Web UI